4.6 项目四：给 Codebase Agent 装上"记忆"和"检索"

4.6 阶段四：给 Codebase Agent 装上"记忆"和"检索"

一、引言

项目背景： 理解了 Agent 的能力上限取决于它能"看到什么信息、遗忘什么信息"后，需要为 Codebase Agent 增加上下文管理、记忆和检索增强能力。

我们把这件事画成一张 **能力供给图**：

代码块

```
              ┌──────────────────────────────────────┐
   任务进来 → │          Codebase Agent              │            │
              └──────────────────────────────────────┘
                                 ▲               ▲
          遗忘了什么?            │               │ 能看到什么?
                                 │               │
         ┌───────────────────────┘               └──────────────────┐
   【长期记忆 OKF】                                          【检索增强 RAG】
   跨 Run 记住"读过的概念"                        改代码前查到"业务规则/为什么"
```

这张图覆盖本节两个机制：**长期记忆**（管"遗忘"）和 **RAG 知识库**（管"看到"）。下面分两大步把它们接到同一个 Agent 上，再收敛到五件事验收：**找得到、装得下、记得准、恢复得了、解释得清**。

尹会生 9600

二、给 Agent 加长期记忆（OKF）

2.1 长期记忆解决 project3 的什么问题

长期记忆要解决的就是让 Agent 把"读过的概念"落成 OKF 文档存下来，**下一轮对话前自动回读**。

2.2 核心提示词做了哪六件事

1. **配置**：在 platform.yaml 配置文件增加开关。
2. **记忆模块 src/memory.ts**：提供 list / search / read / remember / digest / validate 接口。
3. **两件受治理的工具 `src/tools/memory.ts`**：`memory_recall`（risk=read，回读）和 `memory_remember`（risk=write，写入）。它们和十件仓库工具一样注册进 `ToolRegistry`、过 RBAC 与白名单、走 `ToolRuntime` 落审计——**不许绕过治理层**。
4. **运行前自动加载**：`renderSystemPrompt` 在系统提示词末尾追加一段"长期记忆（OKF）"。
5. **提示词规则**：在 `config/prompts.yaml` 的 `codebase.agent` 模板里补一段使用规则。
6. **测试 `tests/unit/kg_memory.test.ts`**：至少覆盖 remember 落盘是合法 OKF 概念、`index.md`/`log.md` 同步、recall 按 query 与 id 取回、`..` 穿越被拒、以及最关键的**跨会话回读**。

2.3 怎么运行

代码块

```
# 1) 记住一个概念（写需要审批）
BODY=$(printf '# 调用链\n路由→Service→仓储（内存假库）。')
   node --import tsx src/cli.ts invoke memory_remember "$(jq -nc --arg b "$BODY" '{id:"concepts/login-flow",type:"Code
 Concept",title:"登录调用链",description:"从路由到假数据库的登录主链",tags:["auth"],body:$b}')"

node --import tsx src/cli.ts approvals decide <approvalId> --decision approve --actor trainer-1 --roles maintainer
   node --import tsx src/cli.ts approvals execute <approvalId>

# 2) 新进程回读（=新 Run）
```

尹会生 9600

```
node --import tsx src/cli.ts invoke memory_recall '{"query":"登录"}'

# 3) 全量测试
npm test
```

长期记忆让 Agent 记住"它自己读过的东西"。RAG 解决**从外部资料库里查**。

三、给 Agent 接 RAG 知识库

3.1 RAG 解决 project3 的什么问题

核心价值是：

1. **改代码前，Agent 能查到业务规则和项目约定。**
2. **RAG 提供修改依据，源码和测试确认实际行为。**

3.2 先实现 RAG 后端

RAG 不是一次写进 Agent 的，它先是一个**独立的后端服务**。

代码块

```
rag/
├── ingest.py    # 入库：文档 → BGE 向量库 + BM25 索引，写 storage/
├── query.py     # 查询：带参数调用，混合检索 + LLM 生成，可 --json 机器可读
└── rag/
    ├── hybrid.py       # BM25（jieba 分词）+ 向量检索，RRF 融合
    ├── local_embed.py # BAAI/bge-small-zh-v1.5 本地向量（512 维）
    ├── answer.py       # 带编号证据的上下文装配与提示词
    └── llm.py          # MiniMax chat
```

尹会生 9600

3.3 先入库，再查询

代码块

```
# 先入库（首次或改语料后）
python ingest.py --overwrite

# 查询：hybrid + LLM，拿到"答案 + 引用来源追溯"
python query.py -q "会话的有效期是多少分钟？" --json --top-k 3

# 只用关键词检索、不调模型：仍返回完整引用
python query.py -q "登录会话有效期" --json --mode bm25 --top-k 2 --no-llm
```

3.4 接入 Agent：`RAG提示词.md` 做了什么

关键规格：

1. **只读工具**：`risk=read`，不改动知识库、不写工作区、不触发人工确认；权限 `rag:read`，授予 `reviewer / developer / maintainer` 三个角色（见 `config/platform.yaml`）。
2. **输出必须同时含"答案"和"带编号的来源追溯"**：`formatRag` 把 JSON 排成模型可读文本——【回答】在前，【引用来源追溯】逐条编号，**编号与答案里的 `[n]` 一一对应，错位即判不合格**。这是可追溯性的硬要求。

3.5 怎么运行

先入库，再跑一个真实业务 Run，让 Loop 自己决定要不要调 `rag_query`

代码块

```
# 0) RAG 先入库（首次或改语料后）
cd ../rag && python ingest.py --overwrite && cd ../project3/v2

# 1) 跑一个业务 Run（--decider real 让模型自主决定是否调用 rag_query）
node --import tsx src/cli.ts agent "登录会话有效期是多少？给我带出处的答复" --no-route --decider real

# 2) 观测：看这次 Run 里 rag_query 确实被调用了
node --import tsx src/cli.ts trace <trace_id>

# 3) 全量测试
```

尹会生 9600

```
npm test
```

3.6 知识库维护：资料和源码打架，听谁的。

- 现象：RAG 查到"会话有效期 30 分钟"，但源码里写的是 20 分钟。
- 为什么会发生：资料库会滞后于代码；模型倾向于相信"看起来权威"的那一份。
- 怎样避免：契约里写明**"资料与源码冲突时要把冲突指出来，不能直接认定其中一方正确"**，并在记忆规则里补"与当前代码冲突时以代码为准并更新记忆"。

任何"检索到的事实"都是**待验证的依据**，不是结论；让代码和测试做最终裁决。

四、本章小结

工程师负责	AI Coding 可以承担	必须拿到的证据
定义信息结构与模块边界	编写模块和接口	数据样例、运行记录
制定上下文与记忆规则	实现装配、压缩、存储	Debug Report、恢复结果
选择检索方案并解释取舍	接入索引、混合检索、Rerank	固定问题集的比较结果
定义来源、版本和权限要求	实现过滤与维护流程	引用核对、版本与权限用例
制定最小规格与验收条件	编码、补测试、修复问题	测试结果与失败分析

1. Agent 的能力上限，取决于它**能看到什么信息、遗忘什么信息**——本节用两个机制分别补这两头。
2. **长期记忆（OKF）** 解决跨会话问题，**RAG 知识库 解决文档参考问题**。两者都**不改代码**，只提供动手前的依据；冲突时以代码为准并指出冲突。
3. 两者都**必须走治理层**：注册进 ToolRegistry、受 RBAC 与白名单约束、经 ToolRuntime 落审计、启动时过契约自检。

尹会生 9600